Three nodes comprise the tree: the root sequence, the debug code action, and the verify fix condition. Because the tree's root is a sequence, the two assistants will continue to work one after another until they both return with success. Both assistants converse on the same thread and yet are controlled in a manner that provides constant feedback.

Run the exercise by loading the file in VS Code, or execute it directly from the command line. The example code has a few minor bugs and problems that the assistants will work through to fix. After the ABT completes running successfully, you can open the `assistants_output/fixed_bug.py` file and verify the results are all good.

We've now seen a couple of ABTs in action and understand the nuances of using silos or conversations. The following section will teach you some techniques for building your own ABTs.

6.5 Building ABTs with back chaining

Back chaining is a method derived from logic and reasoning used to help build behavior trees by working backward from the goal. This section will use the back chaining process to construct an ABT that works to achieve the goal. The following list provides a description of the process in more detail:

1. *Identify goal behavior*. Start with the behavior you want the agent to perform.
2. *Determine the required actions*. Identify the actions that lead to the goal behavior.
3. *Identify the conditions*. Determine the conditions that must be met for each action to succeed.
4. *Determine the mode of communication*. Determine how the assistants will pass on information. Will the assistants be siloed or converse over threads, or is a combination of patterns better?
5. *Construct the tree.* Start by building the behavior tree from the goal behavior, adding nodes for actions and conditions recursively until all necessary conditions are linked to known states or facts.